



Grafos



Prof. Rafael Mesquita
rgm@cin.ufpe.br

O que veremos nesta aula?

- Motivação
- Definição
- Representação
- Propriedades e variações
- Busca em profundidade
- Busca em largura

Motivação

- Diversos problemas computacionais (ou não) podem ser definidos por meio de Grafos
- Teoria bastante estudada na matemática
- Algoritmos relacionados a grafos são fundamentais em computação
 - ▣ Investigação algorítmica de grafos
 - ▣ Da onde vem tamanha importância?
 - Em geral
 - Grafos são utilizados para modelar **conexões** emparelhadas entre elementos de um conjunto
 - Solução de um problema pode estar diretamente ligado à tais conexões
 - Ou, indiretamente, conexões ajudam a compreender melhor o problema em questão

Motivação

□ Exemplos de aplicações [2]

□ Mapas

- Melhor caminho dado um conjunto de pontos e **conexões** entre eles

□ Conteúdo web

- Diversas páginas **conectadas** por meio de links (mecanismos de busca)

□ Comércio

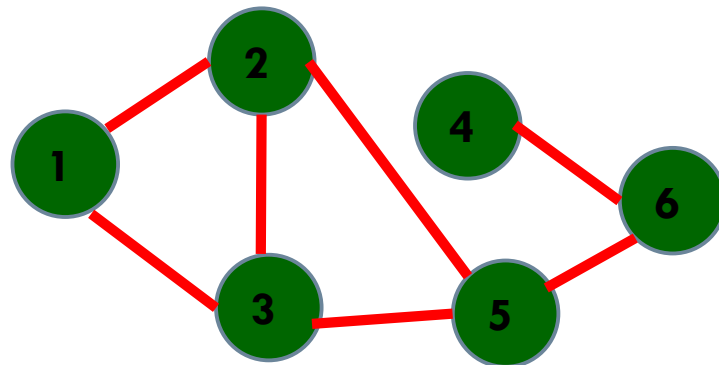
- Pessoas compram e vendem no mercado
- **Transferência** (conexão) de dinheiro é importante para o entendimento do mercado

Motivação

- Exemplos de aplicações [2]
 - ▣ Software
 - **Relacionamento** entre componentes de um software
 - ▣ Redes sociais
 - ▣ Redes de computadores
 - ▣ ...

Definição e propriedades

- Definição de um grafo (G)
 - ▣ Conjunto de vértices conectados por arestas
 - ▣ Vértices (**V**)
 - Elementos do conjunto representado pelo grafo
 - ▣ Arestas (**E** - edges)
 - Conexões entre os elementos (vértices) do Grafo
 - ▣ $G = (V, E)$



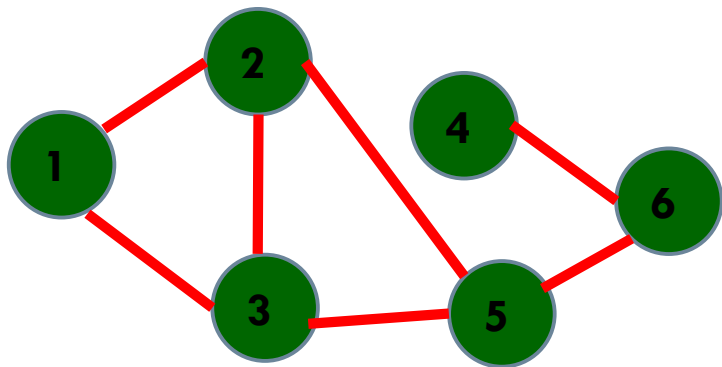
Representação

- Matriz de adjacências
- Lista de adjacências

Representação

□ Matriz de adjacências

□ $G = (V, E)$

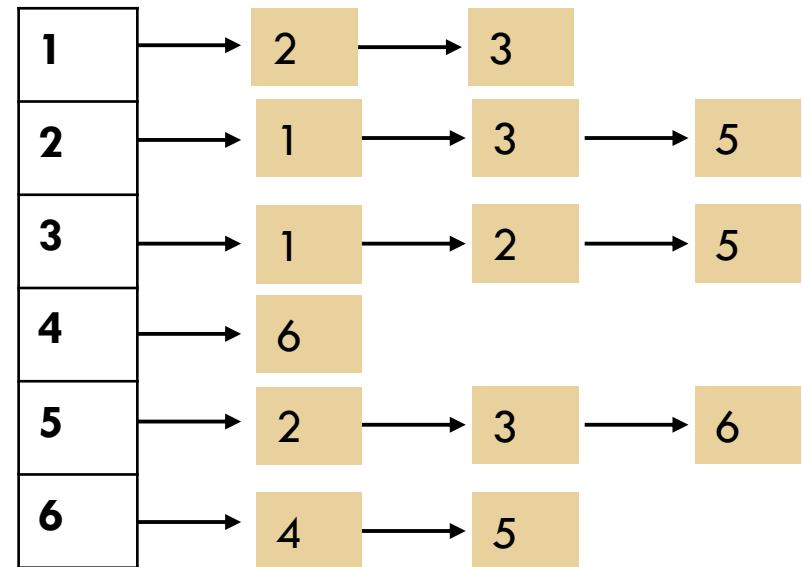
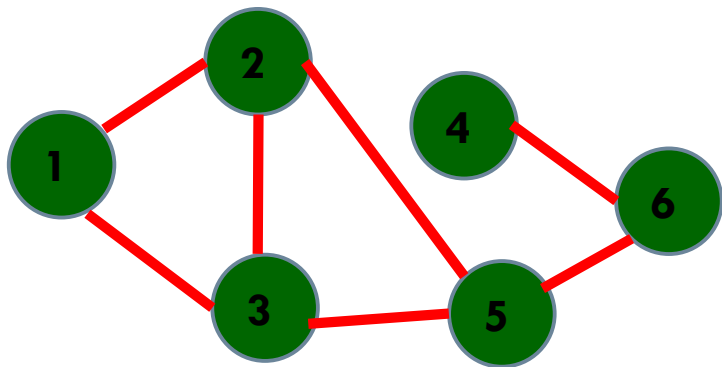


	1	2	3	4	5	6
1	0	1	1	0	0	0
2	1	0	1	0	1	0
3	1	1	0	0	1	0
4	0	0	0	0	0	1
5	0	1	1	0	0	1
6	0	0	0	1	1	0

Representação

□ Lista de adjacências

□ $G = (V, E)$



Representação

- Lista de adjacências normalmente é preferível em relação à matriz de adjacências
 - ▣ Mais eficientes em termos de memória para representar grafos esparsos
 - Se matriz de adjacências for esparsa, um grande desperdício de memória ocorre
 - Grafo esparsa: $|E| \ll |V|^2$
 - ▣ Quando utilizar matriz de adjacências?
 - Grafo denso: $|E| \cong |V|^2$
 - Quando é necessário realizar consultas de forma extremamente rápida
- ▣ Questão similar à decidir quando utilizar listas encadeadas ou vetores

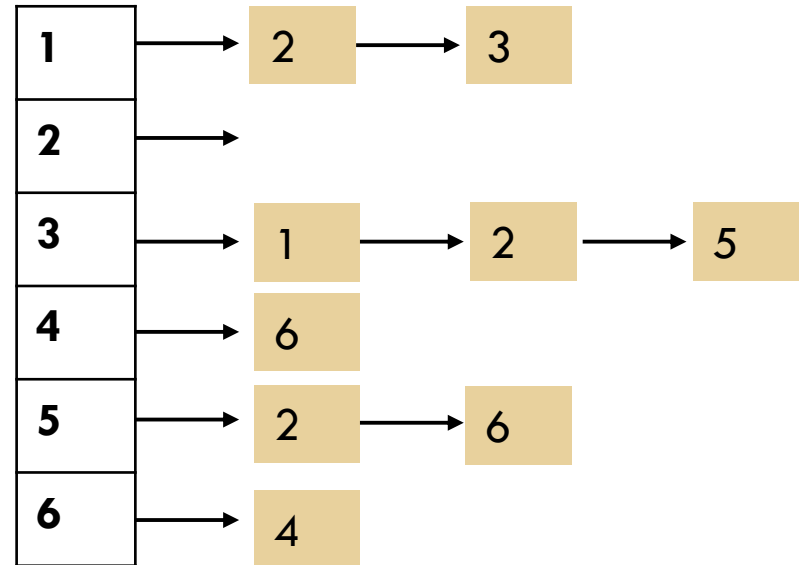
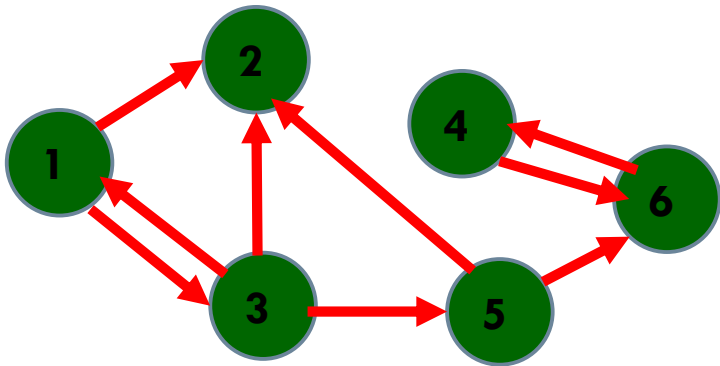
Grafo orientado

- Em grafos não-orientados, se uma aresta (u, v) liga os vértices u e v , isso significa que é possível ir de u para v , e de v para u
 - ▣ $(u, v) = (v, u)$
 - ▣ v aparece na lista de adjacência de u
 - ▣ u aparece na lista de adjacência de v
- Para um grafo orientado para uma aresta (u, v) somente é possível ir de u para v
 - ▣ v aparece na lista de adjacência de u
 - ▣ Opcionalmente, é possível que também exista uma outra aresta (v, u)
 - Então, u apareceria na lista de adjacência de v

Grafo orientado

□ Lista de adjacências

□ $G = (V, E)$



Análise de complexidade de espaço

□ Utilizando listas encadeadas

▣ Grafo orientado

- V vértices
- $|E|$ arestas
 - Somatório da quantidade de elementos em todas as listas de adjacências

▣ Grafo não-orientado

- V vértices
- $2|E|$ arestas
 - Somatório da quantidade de elementos em todas as listas de adjacências
- Quantidade de memória
 - $\Theta(V + E)$

Análise de complexidade de espaço

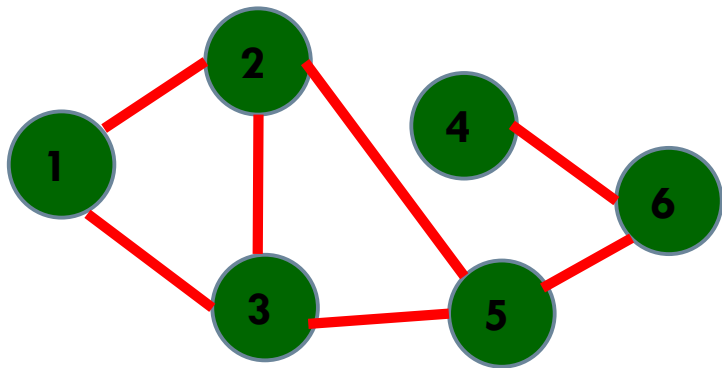
□ Utilizando matriz de adjacências

- Matriz $A_{|V| \times |V|}$

- $\Theta(V^2)$, independentemente do número de arestas

- Caso grafo não seja orientado

 - Matriz é simétrica ao longo da diagonal principal



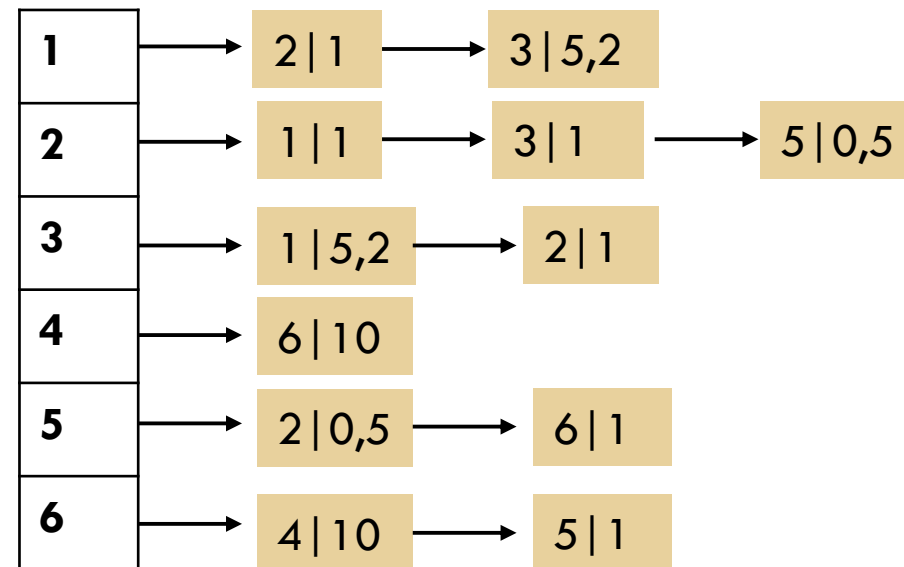
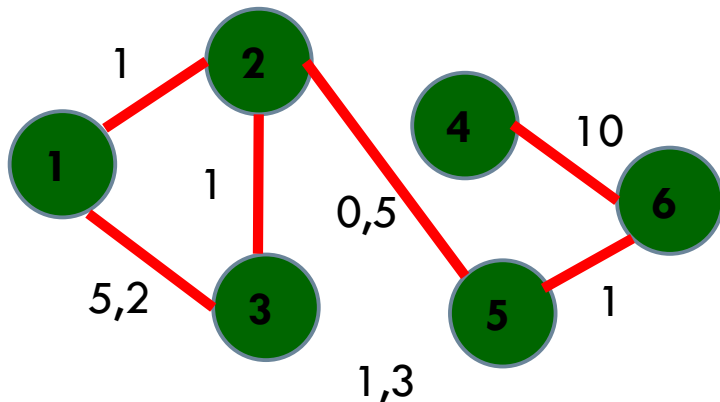
	1	2	3	4	5	6
1	0	1	1	0	0	0
2	1	0	1	0	1	0
3	1	1	0	0	1	0
4	0	0	0	0	0	1
5	0	1	1	0	0	1
6	0	0	0	1	1	0

Análise de complexidade de espaço

- Utilizando matriz de adjacências
 - ▣ Matriz $A_{|V| \times |V|}$
 - ▣ $\Theta(V^2)$, independentemente do número de arestas
 - ▣ Caso grafo não seja orientado
 - Matriz é simétrica ao longo da diagonal principal
 - É possível armazenar apenas os elementos da diagonal principal e os elementos acima
 - Reduz quase pela metade o uso de memória

Grafos ponderados

- Cada aresta pode possuir um peso associado
 - ▣ Função peso $w: E \rightarrow R$
 - ▣ $w(u, v)$ da aresta (u, v) é armazenado com o vértice v na lista de adjacências de u



Busca em largura

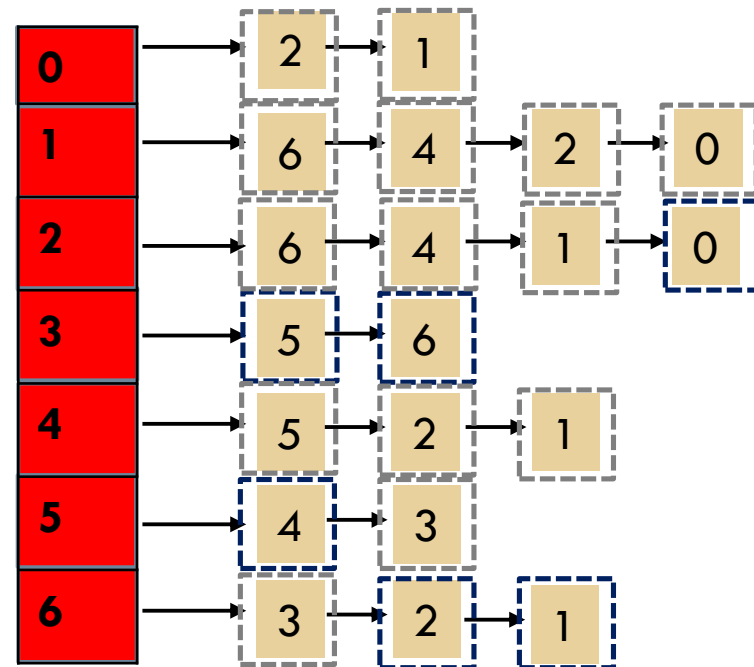
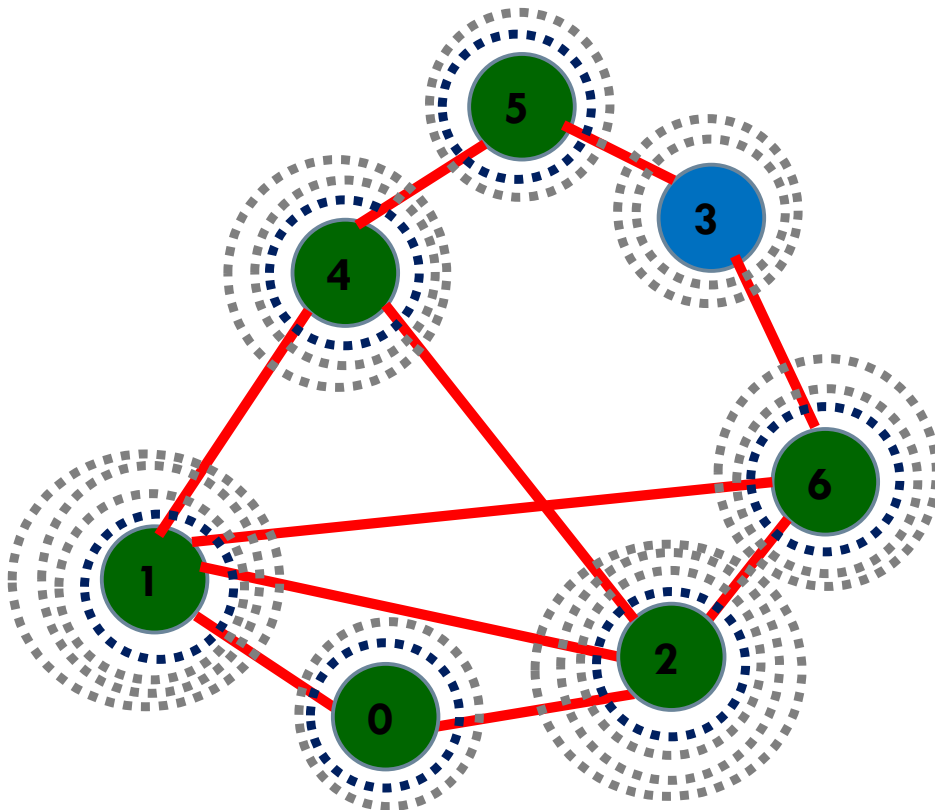
- *Breadth First Search* (BFS)
- Idéia central:
 - ▣ Dado um grafo $G(V, E)$ e um vértice s visitado em um dado instante
 - Descubra todos os vertices à distância 1 de s
 - Em seguida descubra os vertices à distância 2, e assim por diante
 - Todos os vertices à distância k são descobertos antes de quaisquer vertices à distância $k + 1$

Busca em largura

- Ao término da execução, todos os vértices acessíveis a partir de s são descobertos
 - ▣ Para cada vértice z acessível a partir de s , algoritmo encontra caminho mais curto de s para z
 - Já que todos os vertices à distância k são descobertos antes de quaisquer vertices à distância $k + 1$
- Produz árvore primeiro na extensão
 - ▣ Raíz s
 - ▣ Contendo caminho mais curto de s até cada vértice acessível
 - ▣ Caminho também pode ser representado por uma floresta, caso diversas origens sejam consideradas

Busca em largura

□ Exemplo



Busca em largura

- Controle do andamento da busca
 - ▣ Utiliza uma fila para armazenar os vértices que serão visitados
 - ▣ Para fins ilustrativos, cada vértice possui uma cor
 - Branco
 - Nós ainda não descobertos
 - Por padrão todos os vértices são iniciados como brancos
 - Cinza
 - Já descobertos
 - Existem nós adjacentes que ainda não foram descobertos
 - Preto
 - Já descobertos
 - Todos os nós adjacentes já foram descobertos
 - ▣ Na prática, basta marcar cada vértice como sendo visitado ou não
 - Com isso, evita-se explorar mais de uma vez a vizinhança de um vértice já explorado

Busca em largura

- Controle do andamento da busca
 - ▣ Utiliza 3 vetores para armazenar as seguintes informações de cada vértice
 - Cor
 - Vértice predecessor
 - Distância para a origem

Busca em largura

- Árvore primeiro na extensão
 - ▣ Inicialmente contém apenas a raiz (vértice de origem s)
 - ▣ Sempre que um vértice branco v é descoberto partindo do vértice u , v e a aresta (u, v) são adicionados à árvore
 - u é pai de v na árvore

Implementação

```
class Grafo {
private:
    int numVertices;
    ListaEnc** vertices;

public:

    int getNumVertices()
    {
        return numVertices;
    }
    Grafo(int numVertices) : numVertices(numVertices)
    {
        vertices = (ListaEnc**)malloc(numVertices*sizeof(ListaEnc*));
        for (int i = 0; i < numVertices; i++)
            vertices[i] = criaLista();
    }
}
```

Implementação

```
void conecta(int v1, int v2)
{
    if (v1 >= numVertices || v2 >= numVertices || v1 < 0 ||
    v2 < 0)
        return;

    if(!existe(vertices[v1], v2))
        insereLista(&vertices[v1], v2);

    if(!existe(vertices[v2], v1))
        insereLista(&vertices[v2], v1);
}

void imprime()
{
    for (int i = 0; i < numVertices; i++)
        imprimeLista(vertices[i]);
}
```

Implementação

```
void BFS(int s)
{
    if (s < 0)
        s = 0;
    if (s >= numVertices)
        s = numVertices;

    int* cor = new int[numVertices]; // 0=branco 1=cinza 2 ou restante =
    preto
    int* pai = new int[numVertices];
    int* dist = new int[numVertices];

    for (int i = 0; i < numVertices; i++)
    {
        cor[i] = 0;
        pai[i] = -1;
        dist[i] = -1;
    }
}
```

Implementação

```
cor[s] = 1;  
dist[s] = 0;
```

```
Fila* fila = criaFila();
```

```
insereFila(fila, s);
```

Implementação

```
while (!fila_vazia(fila))
{
    int origem = removeFila(fila);

    for (ListaEnc* viz = vertices[origem]; viz != NULL; viz = viz->prox)
    {
        if (cor[viz->info] == 0)
        {
            cor[viz->info] = 1;
            pai[viz->info] = origem;
            dist[viz->info] = dist[origem] + 1;
            insereFila(fila, viz->info);
        }
    }
    cor[origem] = 3;
}
for (int i = 0; i < numVertices; i++)
    printf("\npai: %d dist: %d", pai[i], dist[i]);
} //fim BFS
```

Implementação

□ Exemplo

```
int main()
{
    Grafo grafo(7);
    grafo.conecta(0, 1);
    grafo.conecta(0, 2);
    grafo.conecta(1, 2);
    grafo.conecta(1, 4);
    grafo.conecta(1, 6);
    grafo.conecta(2, 4);
    grafo.conecta(2, 6);
    grafo.conecta(3, 5);
    grafo.conecta(3, 6);
    grafo.conecta(4, 5);
    grafo.imprime();
    grafo.BFS(3);
    return 0;
}
```

```
Imprimindo a lista:
2 1
Imprimindo a lista:
6 4 2 0
Imprimindo a lista:
6 4 1 0
Imprimindo a lista:
6 5
Imprimindo a lista:
5 2 1
Imprimindo a lista:
4 3
Imprimindo a lista:
3 2 1
pai: 2 dist: 3
pai: 6 dist: 2
pai: 6 dist: 2
pai: -1 dist: 0
pai: 5 dist: 2
pai: 3 dist: 1
pai: 3 dist: 1
```

Busca em largura

□ Análise

▣ Fila

- Cada vértice é inserido e removido apenas uma vez
- Complexidade enfileirar e remover
 - $O(1)$
- Complexidade uso da fila
 - $O(V)$

▣ Lista de adjacências

- Lista de cada vértice é percorrida uma vez, apenas quando o vértice é desenfileirado
- Complexidade varredura de todas as listas
 - $O(2E) = O(E)$ para um grafo não orientado
 - $O(E)$ para um grafo orientado

Busca em largura

□ Análise

▣ Inicialização

- $O(V)$

▣ Complexidade total tempo de execução da busca em largura

- $O(V + E)$

- Tempo linear em relação ao tamanho da lista de adjacências

Busca em profundidade

□ Ideia central

- A partir de um nó v que esteja sendo explorado
 - v é marcado para simbolizar que ele já foi explorado
- Em seguida, cada vizinho de v é explorado da forma mais profunda possível
 - Profunda?
 - até que não existam mais vértices vizinhos à visitar em um certo caminho
- Ao encontrar uma ramificação, um caminho é tomado para ser explorado em profundidade e os restantes são percorridos, recursivamente, após o termino do caminho selecionado

Busca em profundidade

- Andamento da travessia
 - ▣ Caminho composto pela busca em profundidade pode ser representado por uma árvore
 - Árvore primeiro na profundidade
 - Ou por uma floresta, caso existam diversas origens para que o grafo seja completamente explorado

Busca em profundidade

- Controle do andamento da busca
 - ▣ Utiliza uma pilha (definida explicitamente, ou implicitamente por meio de chamadas recursivas) para armazenar os vértices que serão visitados
 - ▣ Cada vértice possui uma cor
 - Branco
 - Nós ainda não descobertos
 - Por padrão todos os vértices são iniciados como brancos
 - Cinza
 - Já descobertos
 - Existem nós adjacentes que ainda não foram descobertos
 - Preto
 - Já descobertos
 - Todos os nós adjacentes já foram descobertos

Busca em profundidade

- Controle do andamento da busca
 - ▣ Assim como a BFS, utiliza 3 vetores para armazenar as seguintes informações de cada vértice
 - Cor
 - Vértice predecessor
 - Distância para a origem
 - ▣ Além disso, cada nó é identificado com dois carimbos de tempo
 - Um deles registra quando o nó foi descoberto
 - Enquanto um segundo registra quando sua lista de adjacências foi examinada por completo

Busca em profundidade

```
void DFS()
{
    int* cor = new int[numVertices]; // 0=branco 1=cinza 2 ou restante = preto
    int* pai = new int[numVertices];
    int* dist = new int[numVertices];
    int* dist_fim = new int[numVertices];
    for (int i = 0; i < numVertices; i++)
    {
        cor[i] = 0;
        pai[i] = -1;
    }
    tempo = 0;
    for (int i = 0; i < numVertices; i++)
    {
        if (cor[i] == 0)
            visita_DFS(i, cor, &tempo, dist, pai, dist_fim);
    }
    for (int i = 0; i < numVertices; i++)
        printf("\nvertice [%d] ==> pai: %d dist: %d dist_fim: %d", i, pai[i], dist[i], dist_fim[i]);
}
```

Busca em profundidade

```
void visita_DFS(int vertice, int* cor, int* tempo, int* dist, int* pai, int*
dist_fim)
{
    cor[vertice] = 1;
    (*tempo)++;
    dist[vertice] = *tempo;

    for (ListaEnc* viz = vertices[vertice]; viz != NULL; viz = viz->prox)
    {
        if (cor[viz->info] == 0)
        {
            pai[viz->info] = vertice;
            visita_DFS(viz->info, cor, tempo, dist, pai, dist_fim);
        }
    }

    cor[vertice] = 3;
    (*tempo)++;
    dist_fim[vertice] = *tempo;
}
```

Busca em profundidade

```
int main()
{
    Grafo grafo(6);
    grafo.conecta(0, 2);
    grafo.conecta(1, 2);
    grafo.conecta(1, 4);
    grafo.conecta(1, 5);
    grafo.conecta(2, 4);
    grafo.conecta(2, 5);
    grafo.conecta(3, 5);

    grafo.imprime();
    grafo.DFS();
    getchar();
    return 0;
}
```

```
Imprimindo a lista:
2
Imprimindo a lista:
5 4 2
Imprimindo a lista:
5 4 1 0
Imprimindo a lista:
5
Imprimindo a lista:
2 1
Imprimindo a lista:
3 2 1
vertice [0] ==> pai: -1 dist: 1 dist_fim: 12
vertice [1] ==> pai: 5 dist: 6 dist_fim: 9
vertice [2] ==> pai: 0 dist: 2 dist_fim: 11
vertice [3] ==> pai: 5 dist: 4 dist_fim: 5
vertice [4] ==> pai: 1 dist: 7 dist_fim: 8
vertice [5] ==> pai: 2 dist: 3 dist_fim: 10_
```

Busca em profundidade

- Análise

- Cada vértice é acessado uma vez

- $\Theta(V)$

- Para cada vértice v , todas as suas arestas são acessadas

- $\Theta(E)$

- $\Theta(V) + \Theta(E) = \Theta(V + E)$

BFS x DFS

- Busca em largura permite definir a menor distância de um determinado nó de origem para todos os outros nós acessíveis
- Busca em profundidade é útil, em geral, para extrair informações sobre a estrutura do gráfico
 - ▣ Alguns exemplos:
 - Ordenação topológica
 - Detecção de componentes conectados (BFS ou DFS)
- Qual é mais eficiente em termos de memória?
 - ▣ Depende de se o grafo é “mais largo” ou “mais profundo”
 - ▣ No geral, DFS é preferível

Aplicações

- Aplicações: Ordenação Topológica
 - ▣ Específico para grafos acíclicos orientados
 - ▣ Ordenação linear dos vértices de tal forma que as relações de dependência (ou precedência) entre os vértices seja representada
 - ▣ $a \rightarrow b$
 - b depende de a
 - ▣ Algoritmo
 - Executar DFS(G)
 - À medida que é detectado o tempo de término de cada vértice, inseri-lo à frente em uma lista encadeada

Aplicações

- Aplicações: detecção de componentes conectados
 - ▣ Divide os vértices do grafo em classes equivalentes (componentes conectados)
 - ▣ Aplicação: processamento digital de imagens e visão computacional
 - ▣ Implementação
 - Método DFS() modificado
 - Criação de um vetor de rótulos
 - Durante a varredura de vértices, à cada vértice v não explorado, o valor do rótulo atual é incrementado e passado para todos os vértices adjacentes que são visitados recursivamente durante a exploração da vizinhança de v
 - Ou seja, o mesmo rótulo é atribuído à v e a todos os nós da sua lista de adjacências
 - Nós de mesmo rótulo pertencem ao mesmo componente conectado

Aplicações

- Aplicações: detecção de componentes conectados
 - ▣ Divide os vértices do grafo em classes equivalentes (componentes conectados)
 - ▣ Aplicação: processamento digital de imagens e visão computacional



*Obs: imagem meramente ilustrativa. Somente detectar componentes conectados não é suficiente para vários tipos de problemas!

Aplicações

□ Aplicações: detecção de componentes conectados

```
int main()
{
    Grafo grafo(13);
    grafo.conecta(0, 6);
    grafo.conecta(0, 2);
    grafo.conecta(0, 1);
    grafo.conecta(0, 5);
    grafo.conecta(3, 5);
    grafo.conecta(3, 4);
    grafo.conecta(4, 5);
    grafo.conecta(4, 6);
    grafo.conecta(7, 8);
    grafo.conecta(9, 11);
    grafo.conecta(9, 10);
    grafo.conecta(9, 12);
    grafo.conecta(11, 12);
    grafo.imprime();
    grafo.DFS();
    getchar();
    return 0;
}
```

```
vertice [0] ==> pai: -1 dist: 1 dist_fim: 14 rotulo: 1
vertice [1] ==> pai: 0 dist: 10 dist_fim: 11 rotulo: 1
vertice [2] ==> pai: 0 dist: 12 dist_fim: 13 rotulo: 1
vertice [3] ==> pai: 4 dist: 6 dist_fim: 7 rotulo: 1
vertice [4] ==> pai: 5 dist: 3 dist_fim: 8 rotulo: 1
vertice [5] ==> pai: 0 dist: 2 dist_fim: 9 rotulo: 1
vertice [6] ==> pai: 4 dist: 4 dist_fim: 5 rotulo: 1
vertice [7] ==> pai: -1 dist: 15 dist_fim: 18 rotulo: 2
vertice [8] ==> pai: 7 dist: 16 dist_fim: 17 rotulo: 2
vertice [9] ==> pai: -1 dist: 19 dist_fim: 26 rotulo: 3
vertice [10] ==> pai: 9 dist: 24 dist_fim: 25 rotulo: 3
vertice [11] ==> pai: 12 dist: 21 dist_fim: 22 rotulo: 3
vertice [12] ==> pai: 9 dist: 20 dist_fim: 23 rotulo: 3
```

Referências

1. Introduction to Algorithms, Third Edition, Thomas H. Cormen
2. Robert Sedgewick and Kevin Wayne.
2011. *Algorithms* (4th ed.). Addison-Wesley Professional

